

Supplementary Material for LatticeGuard: Law-Level Metamorphic Regression Oracles for Access-Control Policy Evaluators

Anonymous

TABLE I
HOW TO READ EACH EVIDENCE CLASS.

Class	Supports	Does not imply
Applicable obligations	Declared laws hold for evaluated fragments	Complete language coverage
Rejected candidates	Invalid oracle rows remain visible	Backend failures
Unsupported fragments	Adapter boundaries are explicit	Evidence about unsupported semantics
Seeded drifts	Failure and minimization paths are live	Production defects
Bounded law checks	Relation predicates survive finite falsification	Universal language proof
Trace replay	Reported counts can be recomputed	Scale as the contribution

TABLE II
TERMINAL ACCOUNTING IN THE CURRENT REPLAY.

Row family	Count	Meaning
Applicable obligations	15840	Counted backend-law checks
Primary passes	15840	Law invariant held in counted rows
Primary failures	0	Real violations observed in counted rows
Rejected transformations	3240	Invalid law candidates, not counted
Unsupported fragments	360	Outside declared adapter boundary
Seeded-drift rows	36960	Positive controls, kept separate
Minimized witnesses	5040	Compact replay material for killed controls

I. EVIDENCE BOUNDARY

The main paper carries the scientific claim: access-control laws can be compiled into applicability-checked metamorphic obligations, and those obligations give a safer regression oracle than point examples, differential disagreement, or unrestricted mutation. This supplement expands the evidence boundary behind that claim. It should be read as a companion to the method, not as a place where the contribution is hidden. The central definitions, results, limitations, and novelty argument remain in the main paper.

The evidence boundary is deliberately conservative. LatticeGuard is not a bug-counting study and does not claim that Casbin, Cedar, or OPA are bug-free. A clean primary run is meaningful only after three other facts are visible: invalid transformations are rejected before accounting, unsupported language fragments remain outside the denominator, and the counterexample path is exercised by semantic-drift controls. This supplement therefore focuses on the questions that a skeptical reviewer is likely to ask after seeing a zero-primary-failure result.

This reading rule avoids a common trap in regression-oracle papers. More rows are not automatically stronger evidence. Rows become evidence only after a law predicate says why a before/follow-up pair should preserve or order a decision. Otherwise a generator can create thousands of syntactically valid edits whose expected relation is false. The resulting pass rate may look precise while measuring the generator’s

accidental bias. LatticeGuard makes the admission step part of the result.

II. FORMAL ACCOUNTING SEMANTICS

Let c be a generated candidate for law ℓ . The candidate contains the original policy slice, the follow-up slice, the original request, the follow-up request, and the declared relation family. Three predicates decide whether it can enter the primary denominator: a rejection predicate $\rho_\ell(c)$, an unsupported-fragment predicate $u_\ell(c)$, and an applicability predicate $\pi_\ell(c)$. The predicates are recomputed from the policy and request material; they are not trusted labels from the generator.

$$\text{status}(c) = \begin{cases} \text{rej}, & \rho_\ell(c) \\ \text{unsup}, & \neg\rho_\ell(c) \wedge u_\ell(c) \\ \text{app}, & \neg\rho_\ell(c) \wedge \neg u_\ell(c) \wedge \pi_\ell(c) \\ \text{rej}, & \text{otherwise.} \end{cases} \quad (1)$$

For an applicable row, each declared adapter produces a before/follow-up decision pair (d, d') . The row passes when the law invariant $I_\ell(d, d')$ holds. It fails only when the row is applicable, supported, executable, and violates the invariant. This is a narrow definition, but it is the right one for evaluator regression. Unsupported rows should not be reported as passes; invalid transformations should not become failures; seeded drifts should not be mixed with primary evidence.

The conservative denominator view is included to make overfitting visible. If rejected and unsupported candidates are

TABLE III
LAW-CATALOG EVIDENCE.

Evidence	Count	Role
Relation families	12	Frozen law catalog
Theorem rows	12	Human-readable law statements
Proof obligations	48	Side-condition checks
Bounded core cases	74024	Finite falsification search
Bounded core failures	0	Unsound relations found
Independent kernel cases	432	Adapter-independent replay
Kernel failures	0	Cross-check failures

counted against the run, the pass rate is 81.48%, not 100%. That number is not the primary metric because rejected and unsupported candidates are not valid evaluator obligations. It is a pressure test: it tells the reader that the paper is not hiding the size of the excluded boundary.

III. LAW KERNEL AND PREDICATE SOUNDNESS

The law catalog has 12 relation families. Each relation family contains a law statement, a side condition, a rejection rule, an unsupported-fragment rule, an invariant, and a minimization recipe. The most important design choice is that the predicate must be semantic. It may use syntax to identify the edited material, but admission to the denominator depends on recomputed reachability, matching, closure, dominance, or scope equivalence.

Deny-aware principal monotonicity illustrates the need for this discipline. A naive law might say that substituting a principal with a role-closure superset preserves an allow decision. Under deny-overrides semantics this is false: the larger closure can add a newly matching deny. LatticeGuard therefore admits principal-order rows only when the follow-up closure is a superset and no newly matching deny interferes. The side condition is part of the executable predicate, not a prose exception.

$$\begin{aligned}
 PA(P, q, P', q') \Rightarrow \text{cl}_P(p) \subseteq \text{cl}_{P'}(p') \\
 \wedge \neg \exists r. \text{deny}(r) \wedge \text{match}_{P'}(r, q') \quad (2) \\
 \wedge \mathcal{E}_{\text{Core}}(P, q) = \text{ALLOW}.
 \end{aligned}$$

When the side condition holds, the expected invariant is allow preservation. When it does not hold, the row is rejected rather than counted. The same pattern appears in other laws: a hierarchy closure row must preserve request-relevant reachability; an alpha-renaming row must not touch the requested atoms; a scope split/merge row must preserve the matched action/resource set; a rule-order row must avoid priority-sensitive fragments unless the law explicitly models priority.

These checks are not presented as a complete theorem for all possible authorization languages. They are a falsification-oriented guardrail for this study’s law catalog. If a future

TABLE IV
PREDICATE OBJECTS USED TO KEEP RELATION FAMILIES REVIEWABLE.

Relation family	Predicate object	Rejection boundary
Default deny	Absence of reachable permit	Newly reachable allow
Deny dominance	Matching forbid over allow	Unmatched deny or allow
Forbid precedence	Dominating forbid slice	Backend lacks forbid support
Hierarchy closure	Request-relevant reachability	Closure edge changes query slice
Principal order	Closure superset plus no new deny	New deny becomes reachable
Resource order	Resource closure comparison	Resource path touches request
Alpha renaming	Injective off-slice mapping	Queried atom is re-named
Scope split/merge	Equivalent covered action set	Partition changes matched action
Shadowed removal	Dominated rule witness	Removed rule can decide request
Rule-order invariance	Priority-insensitive fragment	Priority-sensitive backend row
Monotone allow	Permit-preserving extension	Deny side condition violated
Unsupported fragment	Adapter support predicate	Outside declared fragment

relation is added, it should pass the same style of side-condition checks before its generated candidates can affect any primary denominator. That is the scientific point: the method makes law admission reviewable.

Table IV spells out what a reviewer can inspect for each family. The table is intentionally about predicates rather than implementation names. A relation becomes scientific evidence only when the admitted row, the rejected row, and the minimized failure all have a stable semantic object. This is also why the law catalog is not presented as a list of feature toggles. The central claim is that relation families carry their own admission contracts.

This table also clarifies why the bounded evidence is useful even though it is not a universal proof. The finite search does not certify every future policy language extension. It does something more local and reviewable: it checks that the predicates used in this run do not admit the known small countermodels for their own declared relation families. If a relation fails this check, the right response is not to hide the row but to strengthen or remove the relation before it contributes to a denominator.

IV. ADAPTER AND CORPUS BOUNDARIES

The counted run uses two native evaluator roles and one normalized decision harness. Casbin and Cedar are exercised through native evaluator semantics for their admitted fragments. OPA/Rego is exercised as a normalized-decision fragment over a law object whose hierarchy closure is supplied by LatticeGuard’s reference core. That boundary matters. The OPA rows show that the normalized allow/deny decision logic can be driven through Rego under a declared closure

TABLE V
ADAPTER ROLES IN THE EVALUATED FRAGMENT.

Evaluator role	Counted semantics	Boundary
Casbin	Native model, effect, and role matching	Priority-sensitive fragments need explicit support
Cedar	Native permit/forbid decision	Richer entity constraints need new contracts
OPA/Rego	Normalized allow/deny decision over supplied closure	Not a native recursive-closure claim

TABLE VI
SUBJECT STRATA AND EVIDENTIAL ROLES.

Stratum	Count	Primary role
Documentation-derived slices	6	Public semantics and examples
Upstream examples	7	Ecosystem-grounded policies
Native canonical slices	10	Backend fixture linkage
Semantic stress witnesses	96	Dense law-boundary coverage
Generated law probe	1	Canonical corner case

contract; they do not claim that recursive hierarchy closure is implemented natively by the Rego policy in this study.

The corpus is also stratified. Documentation-derived slices and upstream examples provide public semantic anchors. Native canonical slices preserve linkage to real backend fixtures through native self-tests. Stress witnesses deliberately combine relation families that public examples rarely cover densely. The generated law probe exercises canonical corner cases. Collapsing these strata into one benchmark count would make the study look larger while saying less about external validity.

The native self-test layer contains 451 raw decisions with 0 failures before normalization. These self-tests do not prove that every native language feature is covered. They do prevent a weaker failure mode: importing native material, simplifying it into a canonical form, and then forgetting whether the original evaluator would have made the same decisions. In that sense, native self-tests are a provenance check for the corpus, not an additional headline result.

The boundary decisions are deliberately easy to audit. A broader claim would be less useful here: if every unsupported feature were silently folded into the denominator, a clean run could be produced by excluding difficult semantics after the fact. LatticeGuard instead makes exclusions visible. The resulting study is narrower, but the counted obligations have a defensible interpretation.

V. DENOMINATOR AND OVERFITTING CHECKS

A perfect primary pass rate is suspicious unless the denominator is inspectable. The current replay therefore records 14 validity-boundary checks across applicability accounting, unsupported fragments, OPA normalization, holdout coverage, seeded-drift sensitivity, corpus strata, source-boundary sum-

TABLE VII
BOUNDARY DECISIONS FOR EXTERNAL VALIDITY.

Reviewer question	Current decision	Why it is conservative
Are all language features claimed?	No	Unsupported fragments remain outside counts
Are generated probes dominant?	No	Corpus strata are reported separately
Is OPA a full hierarchy engine?	No	Closure is supplied by the reference core
Are native examples decorative?	No	Self-tests bind native decisions first
Can public docs alone explain results?	No	Upstream-only control is measured separately
Can one adapter define truth?	No	Backend decisions are checked against reference semantics

TABLE VIII
VALIDITY-BOUNDARY CHECKS.

Risk	Guardrail
Invalid rows inflate pass rate	Rejection predicate recomputed before accounting
Unsupported rows disappear	Unsupported count remains visible and excluded
OPA boundary is overstated	5280 normalized-decision rows reported under closure contract
Synthetic rows dominate story	Public/native/stress/generated strata remain separate
Failure path is inert	13.64% seeded positive-control kill rate
Baseline comparison is only conceptual	6 same-stream deployed-tool comparisons
Outcome tuning shapes evidence	5148 outcome-independent holdout obligations
Zero failures hide denominator pressure	81.48% conservative pass-rate view

maries, and deployed-tool head-to-head comparison. These checks are designed to catch three families of overfitting: row selection after seeing outcomes, synthetic-source inflation, and denominator manipulation. The 81.48% pressure view is intentionally pessimistic: it keeps 15840 passed obligations in the numerator while counting rejected and unsupported candidates against the denominator rather than silently hiding them.

The holdout evidence is intentionally outcome-independent. It is not a new headline benchmark and does not let the paper claim broader language coverage. Its role is to check that the admission and replay machinery can be applied to material that was not selected to explain the main primary pass count. If the holdout failed admission checks or produced inconsistent accounting, the zero-primary-failure result would be much harder to trust.

The seeded-drift kill rate also needs careful interpretation. The current value, 13.64%, is not intended to be high for its own sake. A positive-control family should exercise different

TABLE IX
DENOMINATOR PRESSURE VIEWS.

View	Value	Interpretation
Primary applicable obligations	15840	Valid evaluator-law rows
Rejected transformations	3240	Invalid candidates kept visible
Unsupported fragments	360	Outside declared adapter support
All-candidate pass view	81.48%	Pressure view, not headline metric
Holdout obligations	5148	Outcome-independent replay surface
Validity-boundary checks	14	Anti-overfitting gate count
Seeded-drift rows	36960	Separate positive-control stream

TABLE X
WHY RELATION-SPECIFIC MINIMIZATION MATTERS.

Failure family	Material that must survive minimization
Deny dominance	Matching allow, matching deny, and request slice
Hierarchy closure	Edge witness and closure-relevant request
Principal monotonicity	Closure superset and no-new-deny side condition
Shadowed removal	Dominating deny and removed allow relation
Alpha renaming	Injective off-slice mapping and unchanged request atoms
Scope split/merge	Equivalent action/resource partition

relation failures without pretending that every seeded row is an equally realistic production bug. A very high kill rate could itself become suspicious if it meant the mutants were trivial. A zero kill rate would mean the failure path is inert. The current result shows that the pipeline detects law-relevant semantic drifts while retaining a conservative interpretation.

The important feature of Table IX is that the paper does not ask the reader to accept a perfect pass rate in isolation. The rejected, unsupported, holdout, and seeded streams remain numerically adjacent to the primary pass count. That adjacency prevents a common overfitting pattern: selecting the denominator after seeing which candidates pass. A future extension can improve coverage, but it must still preserve the same accounting separation.

VI. COUNTEREXAMPLES AND BASELINES

When a seeded drift violates a law, LatticeGuard reduces the case using a relation-specific minimizer. The minimized witness keeps the law name, the before/follow-up decision pair, the request material, and the predicate witness. It removes irrelevant policy material only when the same predicate and invariant violation remain. This is important because a large failing policy is easy to dismiss as an incidental artifact of generation. A small law-named witness is closer to the object a maintainer needs after a release regression.

The baseline design is framed as competing explanations rather than as a generic feature checklist. If native point tests

TABLE XI
ALTERNATIVE EXPLANATIONS TESTED BY THE CONTROLS.

Alternative	What would have to be true
Native and OPA point tests	Same-stream decision checks already explain law failures
Cross-adapter only	Disagreement alone explains semantic regressions
Release-pair precheck	Version comparison alone is enough for regression evidence
Random valid mutation	Syntactic validity is enough for metamorphic testing
No rejection discipline	Invalid transformations do not affect conclusions
No unsupported accounting	Skipped fragments cannot bias the result
Single relation catalog	One law family covers the semantic space
No minimization	Detection alone is sufficient for maintainers
No bounded checker	Empirical passes establish relation soundness

and OPA-style decision tests were enough, before/follow-up law obligations would add little. If cross-adapter disagreement were enough, law-specific applicability would be unnecessary. If release-pair comparison alone were enough, future version pairs would not need predicate admission or replayable witnesses. If random valid-looking mutations were enough, rejection accounting would not matter. If a single relation family explained all seeded drifts, the catalog could be simplified. The deployed crosswalk records 8 familiar idioms, 451 native self-tests, and 5280 comparable differential rows before the component ablations are interpreted. The deployed head-to-head then evaluates point-test and differential idioms as same-stream decision comparisons rather than as adapter-specific test-framework integration: seeded point checks report 13320 decision mismatches, 5040 of those rows become law-level kills after predicate admission, and the point-only remainder stays visible outside the law-failure claim. These alternatives are evaluated over the same frozen evidence stream, which keeps the comparison about oracle design rather than input selection.

This comparison does not claim that LatticeGuard dominates every possible authorization-testing tool on every task. It claims something narrower and stronger: for regression oracles built from access-control laws, the critical difference is whether the expected relation is admitted by a recomputed predicate before evaluation and whether failures reduce to law-named witnesses. Point checks can be stricter at the decision-table level and still be less explanatory at the law level; the baselines are chosen to attack exactly that claim.

The positive-control stream is not used to inflate the main pass rate. It is a calibration device. If the paper reported zero primary failures and no seeded controls, an obvious reviewer concern would be that the oracle is incapable of observing a violation. If the seeded rows were mixed into the primary denominator, the opposite concern would arise: the paper would be counting artificial failures as deployment evidence.

TABLE XII
POSITIVE-CONTROL INTERPRETATION.

Seeded drift family	Expected witness	Reviewer use
Allow stripping	Permit disappears from matched slice	Checks failure path is live
Deny stripping	Dominating deny disappears	Checks deny precedence logic
Hierarchy deletion	Closure edge is removed	Checks reachability predicates
Shadowing change	Dominated rule becomes visible	Checks minimizer keeps cause
Priority disturbance	Rule order changes decision	Checks unsupported boundaries
OPA closure mismatch	Supplied closure differs from policy object	Checks adapter contract

TABLE XIII
HOW THE ABLATIONS ATTACK THE CONTRIBUTION.

Ablation	Attack being tested	Expected consequence
No predicate admission	Law names are enough	Invalid rows enter evidence
No rejection ledger	Exclusions do not matter	Denominator becomes opaque
No unsupported accounting	Adapter gaps are harmless	Unsupported rows masquerade as passes
No native material	Synthetic examples suffice	External-validity anchor weakens
No bounded checker	Empirical rows prove laws	Unsound relation can enter run
No minimization	Detection alone is useful	Witnesses become hard to inspect
Single relation only	One law covers regressions	Multi-causal drifts are missed

Keeping the streams separate is what makes both readings possible without confusing them.

These ablations are stronger than a no-tool comparison because each one is allowed to explain away the same frozen evidence from a different angle. The evaluation therefore asks whether the law-to-obligation design is necessary, not merely whether a named system can produce more rows than handwritten examples.

VII. REPRODUCIBILITY AND RESIDUAL RISKS

The replay surface is deterministic, CPU-local, offline, and anonymous. The reported numeric claims are generated from the replay evidence, and the same evidence is checked for claim consistency, relation coverage, adapter/reference agreement, dependency closure, source linkage, and anonymization. Operational details are kept separate from the paper prose so that the manuscript reads as a research contribution rather than as a laboratory notebook, but the release is still intended to let reviewers recompute the reported quantities.

The strongest remaining limitation is external validity. Authorization languages contain features outside this study’s

TABLE XIV
RESIDUAL RISKS AND CURRENT MITIGATIONS.

Risk	Mitigation and boundary
Law catalog is incomplete	New laws require predicates, side conditions, and replay evidence
OPA scope is overstated	Normalized-decision role and supplied closure are reported explicitly
Deployed baselines are argumentative	Same-stream head-to-head separates point mismatches from law failures
Stress witnesses dominate corpus	Strata are reported separately and not treated as prevalence evidence
Zero failures look overfit	Rejections, unsupported rows, holdout, seeded drifts, and denominator pressure are visible
Formal support is bounded	Bounded replay is presented as falsification support, not universal proof
External systems evolve	Future release-pair claims must re-enter through the same contracts

TABLE XV
CLAIM-TO-EVIDENCE READING PATH FOR A SKEPTICAL REVIEWER.

Concern	Evidence to inspect	What would fail
Perfect pass rate is overfit	Rejected, unsupported, holdout, seeded streams	Hidden denominator manipulation
OPA scope is overstated	Normalized-decision role and closure contract	Native-recursion overclaim
Baselines are ornamental	Component-removal controls	Feature checklist comparison
Formalism is decorative	Bounded and kernel law checks	Unsound relation admission
Corpus is synthetic	Public, upstream, native, stress strata	Single-source benchmark story
Failures are not actionable	Relation-specific minimization	Large unplayable witnesses
Artifact is stale	Claim traceability and replay gates	Paper-number drift

core: obligations, environmental conditions, relationship tuples, recursive data structures, schema evolution, and service-level caching. LatticeGuard’s answer is not to claim those features for free. A new feature must enter as a declared relation contract, adapter support rule, unsupported-fragment boundary, and replayable evidence stream. That design may look slower than unrestricted generation, but it is what prevents the denominator from becoming a hidden modeling decision.

The second limitation is that primary failures are not observed in the current counted run. The paper treats this as a regression-assurance result over declared fragments, not as a discovery claim. In domains where current implementations already satisfy the evaluated laws, the value of a regression oracle is that it creates an inspectable baseline for future edits. The seeded controls and minimizers show that the failure path is not decorative; a future real violation would enter the same schema and reduce to a replayable witness.

The reading path is meant to make rejection easier if the evidence does not support the claim. Each row names a way the paper could be wrong and the evidence surface that would

TABLE XVI
ACCEPTANCE CRITERIA FOR FUTURE EXTENSIONS.

Extension type	Required evidence before counting
New relation family	Law statement, side condition, rejection predicate, invariant, and minimizer
New backend fragment	Adapter support rule, reference agreement, and unsupported-fragment boundary
New corpus source	Public or native provenance stratum and self-test or semantic anchor
New release-pair study	Frozen before/after material and replayable claim ledger
New seeded drift	Declared failure family and relation-specific minimized witness
New baseline	Competing explanation stated before looking at outcomes
New formal check	Bound, domain, and relation family stated explicitly

reveal it. This is the standard the project should preserve after publication as well: adding a new adapter, language feature, or relation family should add a new auditable boundary, not only a larger count.

The review package is therefore best read as a regression-oracle construction rather than a benchmark leaderboard. Its contribution is not that every authorization language has been covered. Its contribution is that an evaluator-law claim can be admitted, rejected, minimized, bounded, and replayed without letting invalid transformations become silent evidence. That is the property future release-pair studies can reuse.

These criteria are stricter than ordinary benchmark growth. A larger corpus or another backend is valuable only if it preserves the claim boundary. The extension must say what is newly supported, what remains unsupported, and how a violation would reduce to a concrete witness. Otherwise growth would make the artifact look stronger while making the scientific interpretation weaker.

This is also the reason the release keeps one current evidence surface rather than multiple named revisions. Reviewer-facing claims should refer to the same frozen replay and the same paper-visible numbers. If a later run changes any denominator, support count, or failure family, the correct response is to replace the evidence surface and update the claims, not to leave parallel versions for reviewers to reconcile.